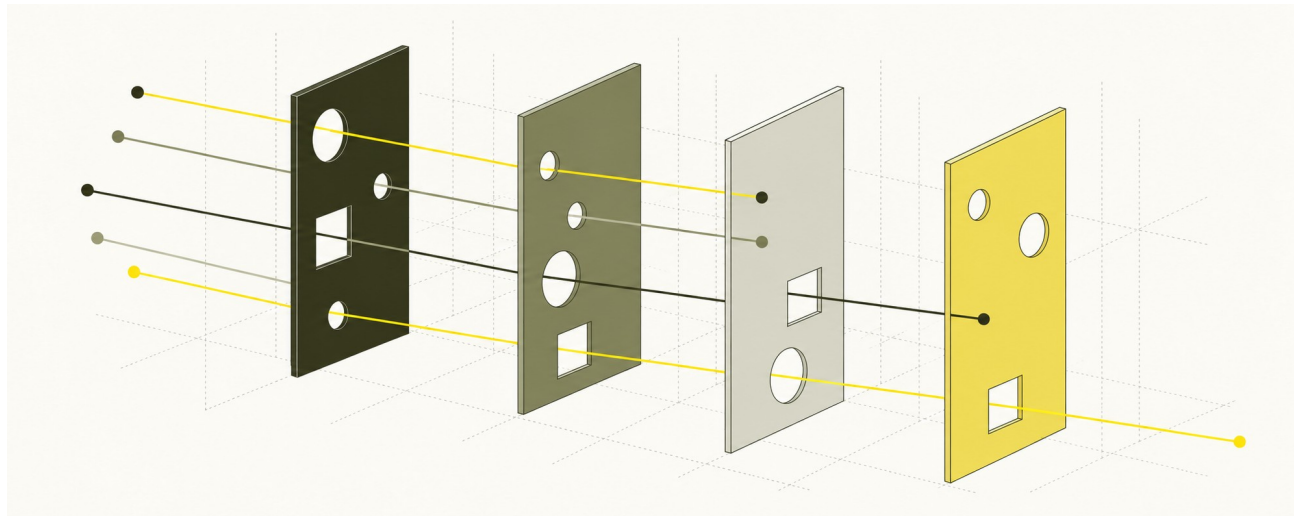


Choosing and Routing Open Moderation Models

A practitioner report

Selecting models and designing moderation workflows
around policy fidelity, accuracy, cost, and latency.



Contributors

ROOST Jeremie Ponak

Musubi Alice Hunsberger and Juliet Jonak

Additional research by Nikki Marinsek

6 October 2026 • Version 1.0

Contents

Introduction	3
Part 1 Model selection	4
Part 2 Routing and cascading between models	10
Reference routing architectures	13
Conclusion	18
Appendix 1 Examples of routing architectures	19
Appendix 2 Model examples from the ROOST Model Community	23

Introduction

No single model is best for Trust & Safety (T&S) work. The right choice depends on the task, where the model sits in your product, how faithfully it follows your policies, and whether its cost and speed allow you to review the right amount of content. This report provides a practical framework for selecting open moderation models and combining them into workflows that fit your team's policies and operating constraints.

In 1996, an entire town in northern England could not sign up for AOL. Residents' registrations were rejected by a profanity filter because a four-letter word appeared within their town's name: Scunthorpe. The mistake became known as the [Scunthorpe Problem](#) and represented state-of-the-art moderation at the time.

Thirty years later, our shared internet spaces have multiplied, and safety teams have more sophisticated options for protecting users from harm, even as broad keyword filtering retains many useful applications. Models designed for safety are proliferating, but choosing the right one has not become easier. Capacity-constrained teams may feel overwhelmed by the seemingly similar options available on [Hugging Face](#) or through the [ROOST Model Community](#) (RMC).

The aim is to avoid a Scunthorpe 2.0: a model that does not follow your community's policies, or whose cost and speed allow you to review only a fraction of your content. Whether you are new to models or already using them, the following sections address the factors that shape this choice.

Part 1 explains how to narrow the field to models worth testing. It considers the accuracy you need, the costs of missed violations and false flags, budget and scale, decision speed, policy changes, language coverage, and expected adversarial pressure.

Part 2 explains how to combine models effectively. Different surfaces and harms have different requirements for speed, cost, policy fidelity, and review, so a single model may not suit every decision. We describe single-pass systems, cascades, and routing by surface or harm, along with thresholds and escalation paths.

Part 1 Model selection

There are many ways to compare moderation models, but they are not equally useful. We recommend starting with four questions:

- How accurate is the model on your policy?
- What does it cost at your scale?
- How fast is it?
- How easily can you steer it as your policy changes?

These four factors usually determine whether a model is viable. Other considerations depend more heavily on your product: language coverage, adversarial robustness, deployment requirements, and the outputs or explanations you need. A model can perform well on the first four factors and still leave gaps in your coverage or use case.

Accuracy

An effective moderation system needs to catch content you want removed without overflagging benign content. False flags add noise and can create substantial costs. No model will be perfect, but you should understand which errors it makes, how often they occur, and whether they are acceptable for the task.

Public benchmarks help narrow the field. [Artificial Analysis](#) compares guardrail models on F1, recall, specificity, latency, and price across three public harms datasets: WildGuardTest, ToxicChat, and XStest. [RakaBench](#) provides a more localized view of moderation performance across Singlish, Chinese, Malay, and Tamil. Together, these resources help identify broad performance differences and language-specific gaps. Neither substitutes for evaluation on your own policy and traffic. That requires running models on your own data and examining their performance.

Key accuracy terms

Recall: Of all content that actually violates the policy, how much did the model catch?

Precision: Of everything the model flagged, how much was actually a violation?

F1 score: A single score that balances precision and recall. Higher is better when both matter.

False positive: Safe content that is flagged.

False negative: Violating content that is missed.

Prioritize the measure that fits the model's role in your workflow:

- For a first pass that routes potentially harmful content, focus on recall. The goal is to catch as much potentially violating content as possible, even if some safe content moves to the next stage. That stage can resolve false positives.
- For a second layer or deeper policy review, focus on precision. False positives become more expensive when they trigger enforcement, human review, or user friction. The model should be more confident that content violates policy before acting.
- For a model that decides on its own, focus on balanced F1. With no later stage to correct errors, you need a reasonable balance between catching violations and avoiding false flags.

Once you know which measure matters most, consider what the model should return. A classifier can provide a score for thresholds or routing, while some generative models can produce a reason alongside the decision. That explanation may help a human review or understand the decision.

Choosing model outputs

Extra outputs affect cost and consistency. In Musubi's testing, open-ended fields such as step-by-step analysis or a general image description were most useful during policy iteration, but also increased variation in threshold scores between runs.

When these fields were not needed in production, removing them could reduce output tokens by approximately 83%, with no loss in recall. Threshold scores moved about two-thirds less.

Targeted fields behaved differently. Asking the model to answer a specific question before deciding, such as "Is an adult's face visible?", improved accuracy and consistency.

Evaluating on your own data

Most teams begin by building a golden set from examples they have reviewed and agreed on. This exercise exposes inconsistent labels and vague policy language. Musubi has found that reviewing disagreements before tuning can reduce a policy's misses by more than half, with no increase in false positives.

Then:

- Calibrate thresholds on your own data before going live.
- Run in shadow mode first, then manually sample production decisions to distinguish model errors from genuine policy gray areas.
- Stress-test boundaries with synthetic examples of tricky categories, exceptions, and near-boundary cases, using real examples as templates.
- Test the whole workflow, not just individual models. Include examples with multiple violations, safe look-alikes, and a realistic share of violating content.

Price and model size

Model cost determines how much traffic your systems can review. An inexpensive model may be viable on every message or upload, while a more expensive model may make sense only for a sample or for escalated cases.

Classifiers, including [BERT-family](#) models such as [RoBERTa](#) and [DistilBERT](#) and other lightweight models, read the text once and output a score or label directly, making them inexpensive. Generative models produce answers word by word, with each word requiring another pass through the model. This accounts for much of a large language model's (LLM's) cost and latency.

Within a model family, cost scales with size. A mixture-of-experts model activates a fraction of its total parameters on each pass, while a quantized model runs smaller than its full-precision weights. The size of the model being run also determines the compute required. A 1–2B-parameter model such as PolicyLM-1.7B fits easily on a CPU. A model with 20B or more parameters, such as [gpt-oss-safeguard](#), needs dedicated inference hardware that a T&S team usually has to request from another team.

Where you spend model budget ultimately matters more than which model you choose. Once you understand the cost of running each model, the next question is where that spending will be most useful. This is why Part 2 focuses on routing. Musubi learned this firsthand: downsizing the routing model while retaining larger models for specialist stages materially reduced cost, slightly increased specialist escalations, and did not reduce recall.

Include downstream human-review costs in your evaluation. Automatically approving clean content is often the largest operational saving, even when the per-call model price is higher.

Latency

Price determines how much traffic a model can cover; latency determines which surfaces it can cover at all. These factors often involve a tradeoff: models fast enough for live surfaces are usually inexpensive, while models worth escalating to cost more per call.

Set the latency budget by surface:

- Live chat, game lobbies, direct messages (DMs), and usernames require decisions before the conversation moves on, generally within tens to low hundreds of milliseconds. Historically, live chat has relied on fast classifiers. Generative LLMs take seconds per check, with latency increasing as the answer grows. Purpose-built models can therefore be useful. Musubi's PolicyLM-1.7B, for example, is small enough to address this latency constraint while reading your policy at classifier speed. This capability also matters for steerability.
- Uploads, profiles, and listings usually tolerate a second or more. Asynchronous and batch review can take minutes to hours. Acceptable timing depends on your users and industry.

Measure end-to-end system time, not only model execution time. Routing between models makes escalated items noticeably slower, and users experience that full delay. A model that averages 80 ms can still produce a 600 ms decision when it sits behind an API. Whether this is tolerable depends on the moderation surface. For live chat, it is another reason a single-pass model is usually the best approach.

Steerability

Steerability has two dimensions. Output control is the ability to define the categories a model returns instead of relying on someone else's taxonomy. Policy fidelity is how faithfully the model follows your rules, including exceptions and carve-outs.

A fixed classifier such as Meta's LlamaGuard offers little of either. PolicyLM provides output control and substantial policy fidelity, though it is less reliable when a policy asks it to reverse an expectation learned during training. Zentropi's CoPE is designed to support both.

Consider a dating site. Flirting and sexual conversation between consenting adults are part of the product, so a fixed "sexual content" category may flag conversations you want to allow. Meanwhile, harms you care about, such as a match immediately pushing to move the conversation to WhatsApp or asking for gift cards, may not appear in a generic taxonomy. A steerable model lets you express that policy: consensual flirting is allowed, unsolicited explicit images are not, and early requests to move off-platform are flagged.

ROOST's posts on [Shieldstral](#), [CoPE-B-A4B](#), and [gpt-oss-safeguard](#) discuss steerability in more detail. A steerable model is only as good as the policy it receives; the RMC's [policy packs](#) offer a starting point. For a public measure, [Zentropi's Policy Steerability Benchmark](#) provides the clearest test: it presents the same content under two policies, one that should flag it and one that should permit it, and scores the fraction of pairs for which the model follows both.

Steerability does not make a model perfectly deterministic. The same policy can behave very differently across sizes and versions within a model family: one may alternate on hard categories, another may be consistently permissive, and another consistently strict.

Test each model against your golden set, even with an identical prompt, and expect to tune policy language for each model. Identical prompts will not bring out the best in every model, and newer versions are not necessarily better at following your instructions. We regularly test across model families, isolating differences due to model size, reasoning, and temperature settings.

In one example, identical policy language produced different results when evaluating whether a minor was accompanied by an adult. Gemini 2.5 Flash and 3.5 Flash-Lite outperformed 3.1 Flash-Lite. The latter accepted anything that could plausibly suggest a child was accompanied: a hand at the edge of the frame, an adult walking in the distance, or a group of children accompanying one another. That version required additional definition to produce consistent results.

Design choices that affect policy fidelity

Steerability extends beyond labels and policy text. Several design choices can materially affect consistency:

Observations: Some decisions depend on a single fact. Ask the model to commit to that fact first, for example, “adult_present: yes | no,” before deciding whether a policy is violated. In one Musubi child-safety test, this reduced disagreements with moderators by approximately 85%.

Violation names: Labels carry meaning for models. More precise names can improve accuracy. “Teen” worked better than “Young Adult,” and “Unaccompanied Minor” better captured the policy’s intent than “Child.”

Moderation action: Design labels around what happens next. A broad “Minor” flag may be too uncertain for automation, while more specific categories such as “Pre-Pubescent” and “Teen” can support different paths for automated action, age verification, or moderator review.

Test steerability much as you test accuracy: run examples against your own policies, invert the policy’s expectations, introduce carve-outs, and change the topic to see whether the model follows the instructions. For examples, ROOST’s head of product, Juliet Shen, has built small sample datasets and libraries in the [Vibecheck](#) evaluation harness.

Language coverage and adversarial robustness

Open models can be assessed along many other dimensions. Two deserve particular attention: multilingual performance and adversarial robustness. Neither is equally important in every deployment. A single language and limited adversarial pressure may be acceptable in some settings. If users may speak other languages or try to circumvent guardrails, such as within a chatbot, these dimensions need explicit testing.

Multilingual performance

“Multilingual” on a model card does not mean your policy performs equally well in every locale. Test each language, separating capability from calibration. In ROOST’s French probe of PolicyLM-1.7B, the model ranked harmful and safe posts almost equally well in English and French. French scores ran systematically higher, though, and the best threshold was 0.30 in English and 0.20 in French. At a shared threshold, French posts describing recovery were overflagged.

Machine translation can also move scores by itself; a euphemism may become explicit in translation. Use test sets reviewed by native speakers where possible. Language-specific allowlists and blocklists can provide an inexpensive patch for terms the base model under-catches.

Adversarial robustness

The time you invest in robustness should reflect the stakes. If adversarial pass-through would have severe consequences, such as a model gating child sexual abuse material (CSAM) generation, invest substantial testing effort. Monitoring message boards may not require the same level of effort.

Adversarial robustness concerns whether content can persuade a model to change its decision. For any model that reads policy, test policy injection first: fake policy text embedded in the content. In ROOST tests, a forged “policy update” cut recall roughly in half. Injected text placed before the content caused more damage than text placed after it.

Part 2 Routing and cascading between models

Not every model fits every purpose. Sometimes the best option is to route content from one model to another in a cascade. This section explains when to route, what to consider, and how different architectures work in practice.

Not every moderation task is a good candidate for full automation. Today, a mix of classifiers and LLMs can realistically automate profile and bio moderation, user-uploaded images, listings and marketplaces, creator uploads, and spam and scam signals on structured events. Policy-aware live chat is newly realistic; until recently, a fixed classifier was the only option at that speed. Agent-to-agent safety, video with sparse frame sampling, audio, and identification of AI-generated content are emerging but often viable. A human should always remain in the loop for high-severity, high-risk categories such as reports of offline harms.

The choice between a single model and routing depends on the content, policies or harm types, latency and cost constraints, volume, and the complexity your team can manage.

When to route or cascade

Routing is useful when:

- Most content is obviously safe. A small, inexpensive first-stage model clears that content, while a more expensive model, often a reasoning model, examines potential violations. Measure the share of nonviolating content before launch, because the volume sent to the second model directly affects total cost. If 20% or less of content is potentially violating, this is a good choice.
- Your taxonomy includes hard subcategories that benefit from specialists, such as minor presence, CSAM-adjacent content, or self-harm. Reporting workflows may already use these categories, helping direct content to different models and policies.
- Your budget cannot support a large model on all traffic. Identify where false positives or lower accuracy are most costly and route those cases; send the rest through a smaller model.
- Only a minority of decisions needs a written rationale. Route those cases to a model that produces a reason or analysis, and use a labeling or classification model for the rest.

Before splitting a policy into specialist stages, check model size. In Musubi's testing, a single policy with many categories on a capable model performed as well as separate policies. Splitting it only added maintenance. Smaller models behave differently: as a policy grows, they begin missing subtle categories first, while severe categories remain more reliable. Split out or escalate only those subtle categories.

When to use a single model

A single model is useful when:

- Decisions must return immediately in one pass, as in live chat. Serious labels can still receive further investigation, but tight latency requirements mean that work must happen asynchronously.
- Volume is low enough that operational simplicity outweighs the savings. Use one capable, general-purpose model.
- Policies are tightly coupled and require reasoning across categories that a router would separate.

- Your team does not yet have the golden-set discipline needed to tune handoffs safely.

Teams often start with low-complexity workflows that deliver early gains, then build from there. Set the latency budget and the cost of a missed violation first. These decisions constrain the rest: an extremely short decision window rules out routing to a reasoning model, while a strong need to avoid missed violations makes very high recall the first priority.

Routing practices

Tune the first pass for recall

In a tiered workflow, content cleared by the first pass is never reviewed again. False flags at that stage cost only an additional call, so favor flagging. In Musubi's testing, nearly all misses originated in the first pass; it was rare for a specialist to clear an item and drop a second flag. A first pass that returns a single label cannot carry a second violation forward at all.

Musubi found that using a high-recall first-pass policy reduced routing costs by more than half while increasing manual moderator reviews by only 2.5%. Wording matters. Removing exceptions from the first layer and explaining that a miss costs more than an overflag helped keep results consistent, even on smaller models.

Match routing to the available outputs

Where possible, route using output scores and thresholds as well as labels. A classifier's label identifies which category triggered, while its score indicates confidence. Check what the model actually returns before designing routing:

- A label: [CoPE-B](#) returns a single 0/1 token, with no score.
- A score per question: [Shieldstral](#) answers one yes/no policy question per call, with a score you can threshold.
- A score per label: PolicyLM-1.7B returns a 0–1 score for multiple labels in one combined pass.
- A label with reasoning: [gpt-oss-safeguard](#) returns a label and a reason. Route on the label; use the reason for review and appeals.

Return a list of violations rather than a single label. A single label catches violating content just as well but cannot report a second violation. On smaller models, requesting a score per category makes the model consider each category and provides a threshold for each one.

When routing with labels and scores, labels should determine where an item goes, while the score determines whether it goes anywhere.

Use thresholds to manage escalation

If a model outputs scores, use them to tune the volume sent for escalation:

- Below a low threshold, allow the content.
- Above a high threshold, act automatically, for example by blocking, hiding, or muting.
- Between the thresholds, escalate to a larger model or a human. A wider band increases escalations and cost but reduces errors reaching users.

Log every score, not only the label. Scores let you test alternative thresholds without rerunning traffic. Their distributions also provide an early warning of drift; fewer high-confidence scores are a reason to investigate. Adjust thresholds for different scenarios rather than applying them uniformly.

Check the score distribution before designing an escalation band. Some models rarely return intermediate scores. In those cases, an “unsure” band may escalate nearly every flag and allow everything else through, making it ineffective as a cost control. With generative models, policy wording can shape the distribution and clarify what scores mean for automation thresholds.

For example, some platforms treat anyone of ambiguous age as a possible minor. When instructed to flag ambiguous cases, a model may flag them and express its uncertainty in the score. This becomes a problem if a clear violation and an ambiguous image receive the same high score. Musubi found that guidance linking each end of the scale to observable features made scores more consistent. Top scores then reliably represented clear violations suitable for automated handling or high-priority escalation.

Calibrate thresholds for the context

Set thresholds by severity. Classifiers often provide preset confidence thresholds for each category. Rare, severe harms typically warrant a lower threshold so they escalate early, while lower-severity harms such as profanity warrant a higher threshold to reduce unnecessary escalation. [Roblox's PII classifier](#) follows this pattern: it recommends 0.10 for the higher risk of steering users off-platform, compared with 0.60 for asking for personally identifiable information (PII) and 0.55 for disclosing it.

Set thresholds by language. Where performance is lower, use a wider middle escalation band. Where it is higher, fewer cases need a larger model or human review.

Set thresholds by position in the stack. Tune a first pass for recall, then tune the next stage for precision so it can dismiss false positives and examine policy nuance. A standalone model needs a balance of recall and precision. A recall-tuned model on its own can flood the queue; a precision-tuned model on its own can miss too much.

Set thresholds by model. Scores are not comparable across models: a 0.7 from one model is not equivalent to a 0.7 from another. Recalibrate when changing models or editing policy text.

Maintain policies and production controls

Count the policies you must keep in sync. Every router, screen, and specialist adds another policy text that must agree with the others. When adding a category, update the router or screen as well as the specialist. Old carve-outs in a router can quietly prevent a new category from ever reaching the specialist.

Use relevant context. Routing can combine classifier scores with information the model does not see, such as account age, prior enforcement, or surface, including DM versus public post. For example, you might escalate a middle-band score from a new account while allowing the same score from a long-standing account.

As routing becomes more complex, systems and software for managing components and overseeing production become more important. Version policies and routing rules so you can quickly revert when needed. Regularly evaluate sampled production data to identify new errors or drift.

Reference routing architectures

The following architectures illustrate ways to combine models, classifiers, and policies. Appendix 1 applies them to specific use cases.

Deterministic precheck followed by a model

Hash, URL, or keyword matches block content before a model runs. This is inexpensive and fast, and sensitive matched content never reaches a third-party model.

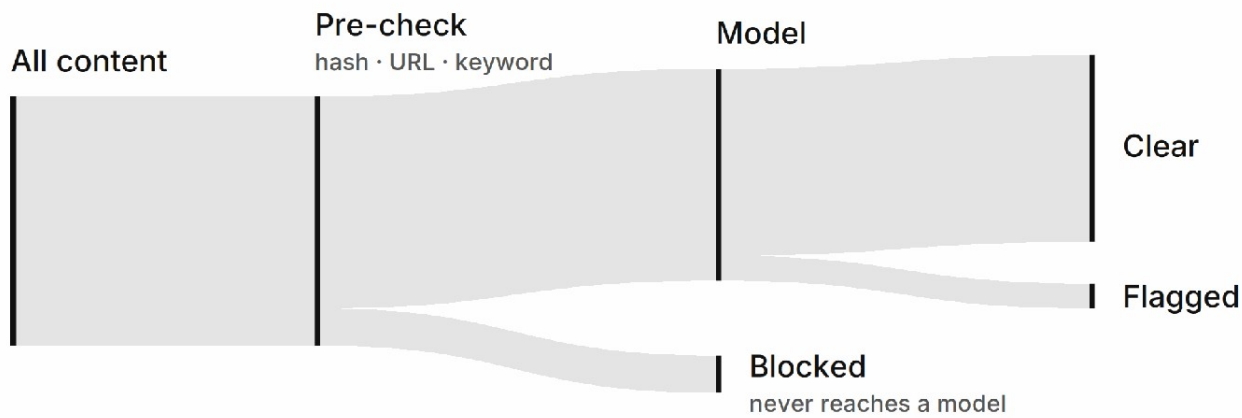


Figure 1. Deterministic precheck followed by a model

Screen then escalate

A small model decides only whether to flag or clear content. Flagged items go to a larger model with the full policy. In Musubi’s testing, this was much cheaper than using the large model on everything and better at reporting every violation on items containing several.

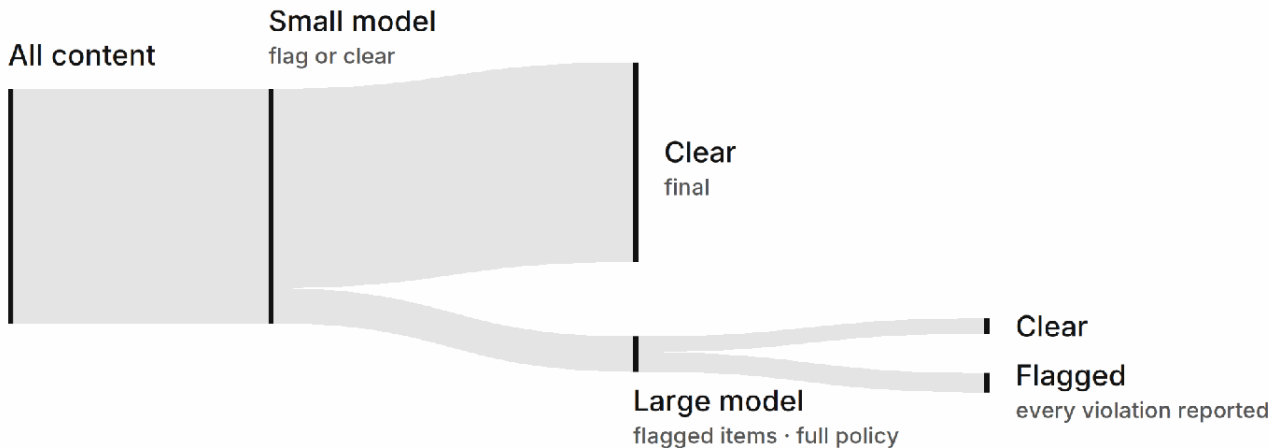


Figure 2. Screen then escalate

Small model with escalation for uncertainty

A small model applies the full policy to all content; only uncertain or severe items escalate. This was also much cheaper and caught slightly more, but the small model’s confident decisions are final, including its misses.

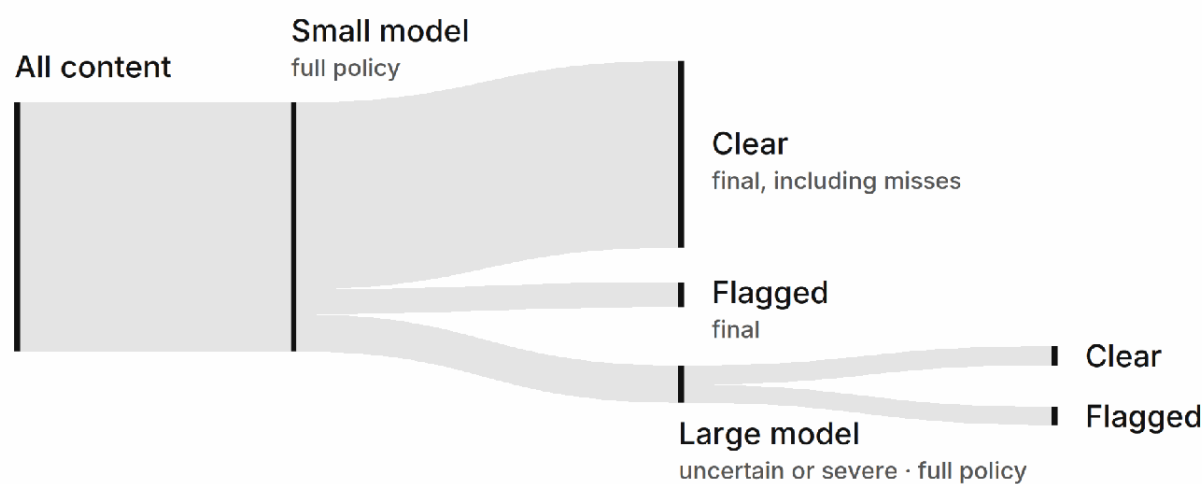


Figure 3. Small model with escalation for uncertainty

Classifier with a scoring head

Add a simple scoring head to a classifier that returns scores for multiple categories. This converts scores into decisions without individual category thresholds and improves performance by accounting for interactions across categories, such as when adding profanity to teasing should escalate it to bullying.

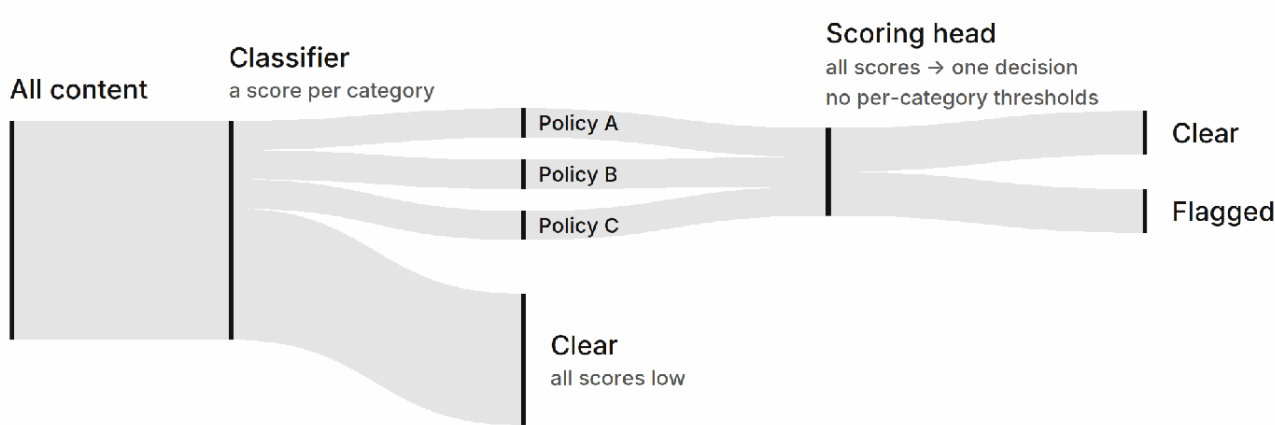


Figure 4. Classifier with a scoring head

Label then judge

A fast labeler produces labels rather than decisions. If no label triggers, automatically approve the content. If a label triggers, a policy-aware reasoning model reads the labels, policy, and context. One label set can feed several policies.

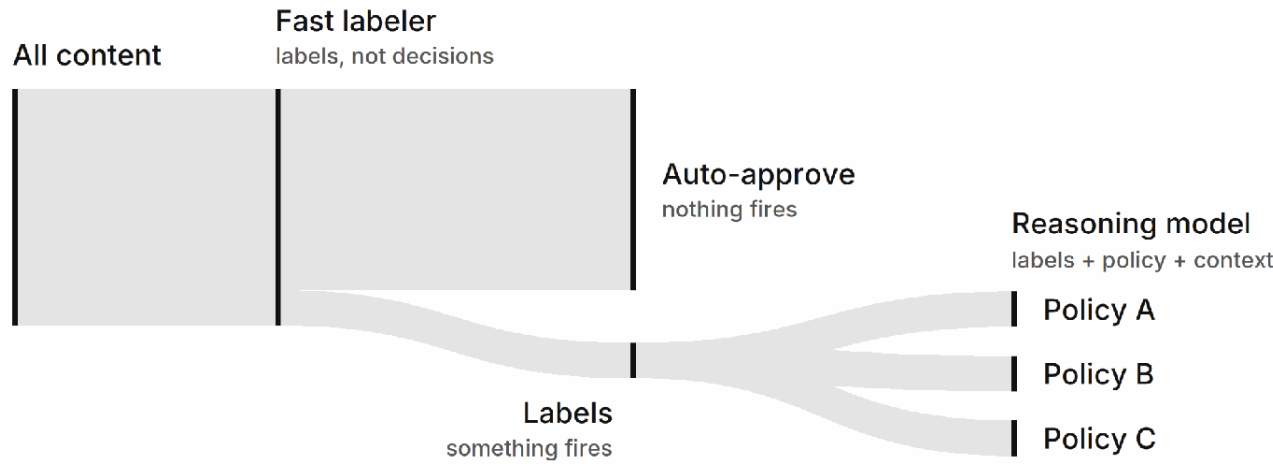


Figure 5. Label then judge

Router policy followed by specialist policies

Use the same model family and route between prompts rather than models. Tune specialist policies for hard subcategories. Preserve signals across each handoff: if a specialist returns “clear,” do not discard other legitimate flags raised by the router.

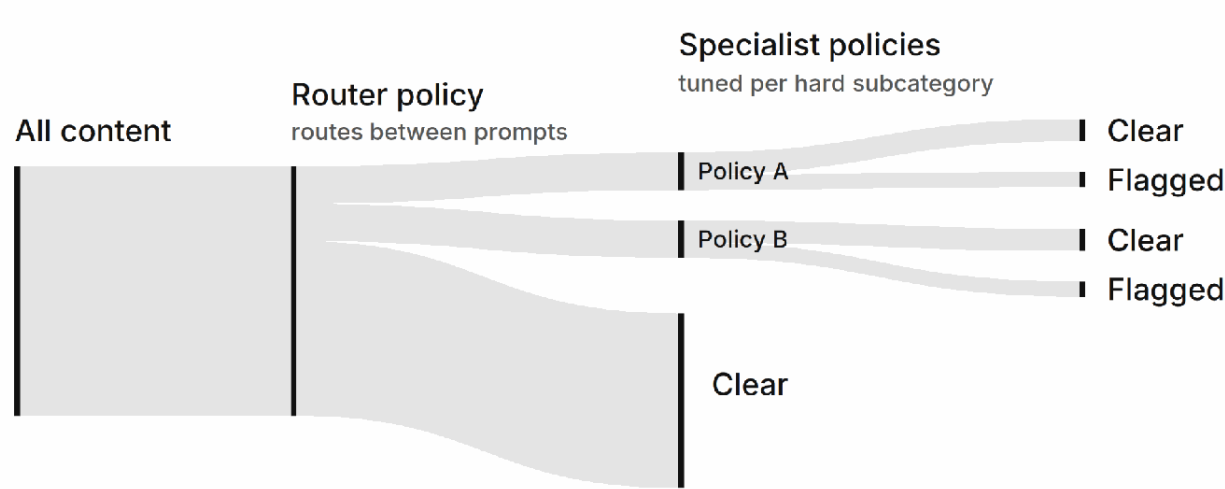


Figure 6. Router policy followed by specialist policies

One check per category

Run a small yes/no policy for each category in parallel. In our testing, this caught the vast majority of violations, but each check saw only its own category and therefore overflagged safe content. It also cost more and increased the number of policies to maintain.

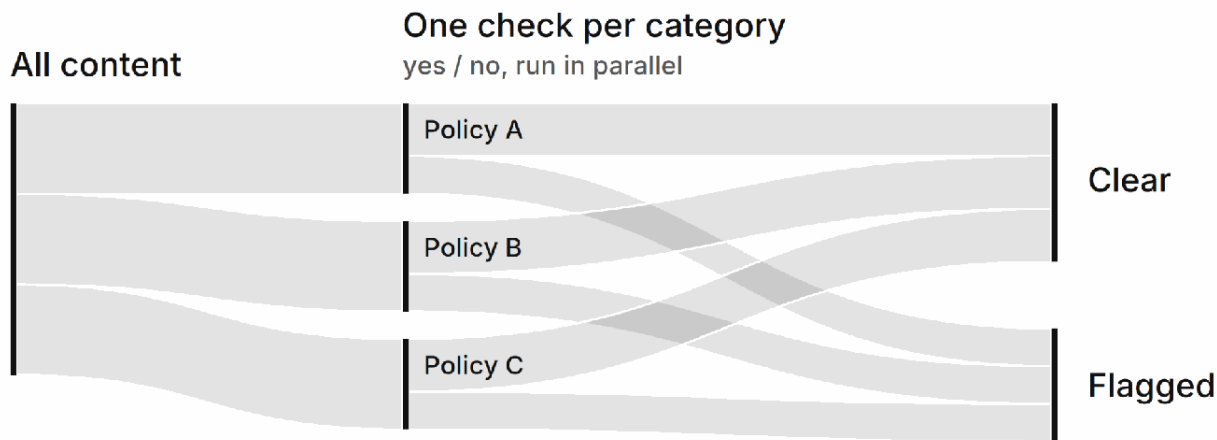


Figure 7. One check per category

Behavioral signals followed by content review

A model using structured signals handles straightforward decisions. Ambiguous cases go to a content model or a human.

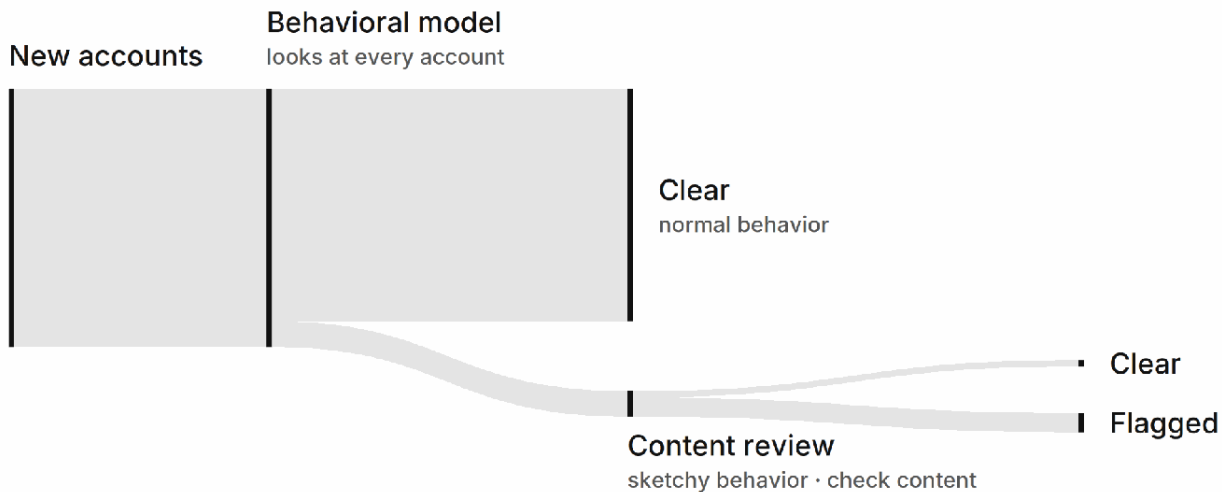


Figure 8. Behavioral signals followed by content review

Content review followed by behavioral signals

A content model identifies potentially scam-related or suspicious content, then routes it to a larger model for a holistic review of the account and its behavior.

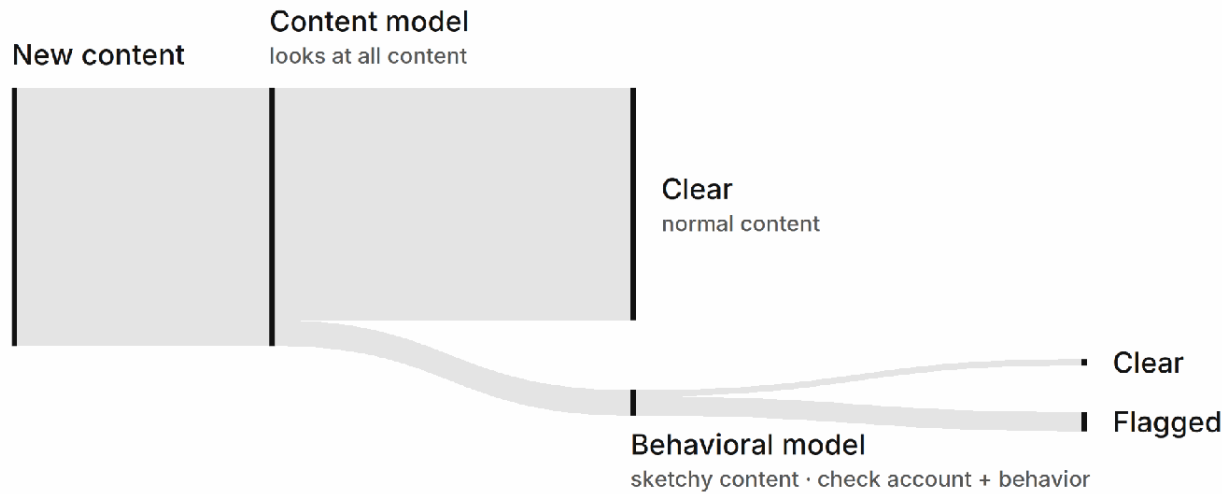


Figure 9. Content review followed by behavioral signals

Provider fallback

Use a primary model, with a secondary model as fallback when an error, timeout, or quota limit occurs.

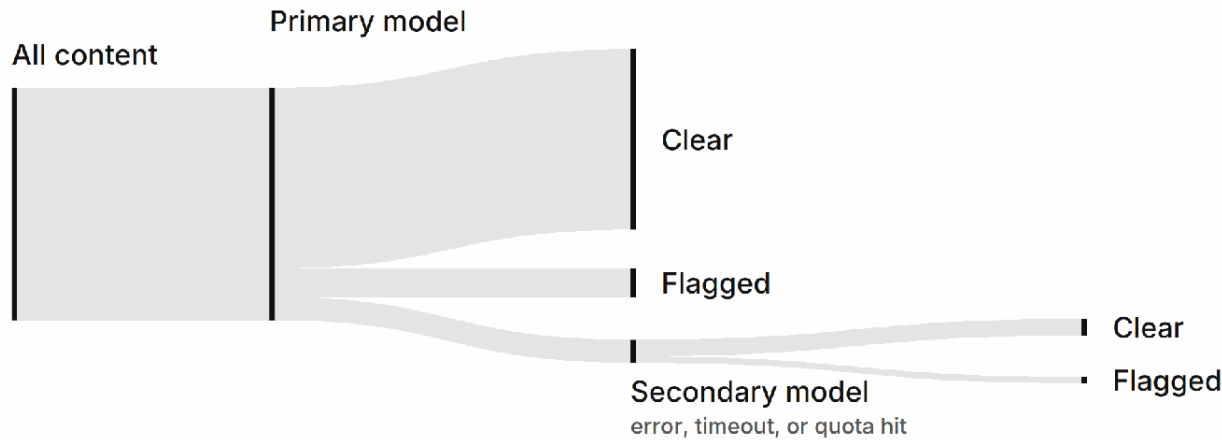


Figure 10. Provider fallback

Human review as the final tier

Stacking models for straightforward decisions makes human judgment affordable at scale.

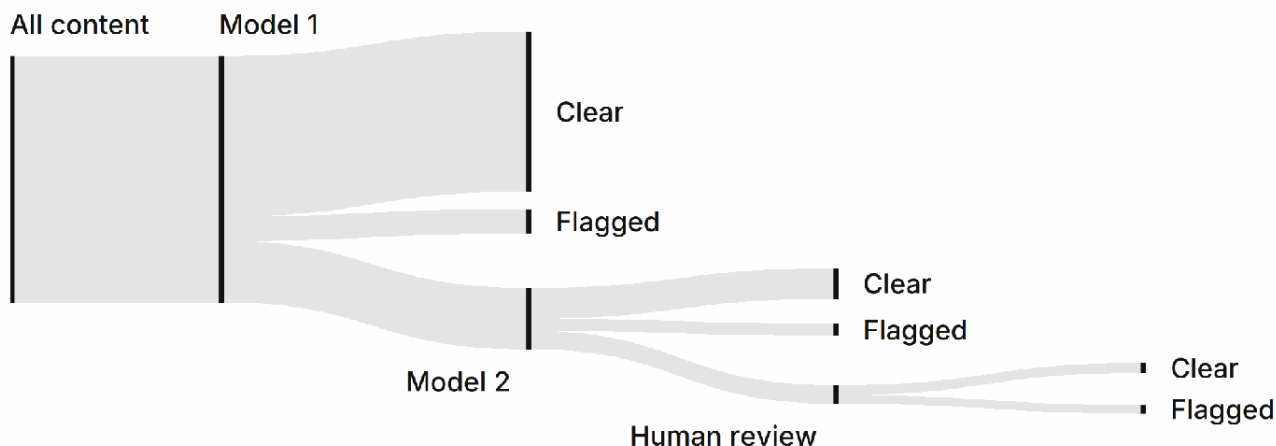


Figure 11. Human review as the final tier

Conclusion

Trust & Safety enforcement used to be constrained by what a model had been trained on. With policy-steerable models, outcomes depend on the model, the policy text, and how they are connected. This gives T&S teams influence over the whole system as well as the policy document, but adds complexity and potential failure points.

Start simple, and add complexity when cost or latency requires it. Test early and often with golden datasets and production data so you understand what is changing and why. Spend model budget where it meaningfully improves outcomes. The largest model is not needed everywhere; small, open-source models are often sufficient.

AI changes quickly, which is why this report keeps its guidance as general and broadly applicable as possible. Model-specific findings appear as illustrative examples: results vary by model, policy, and platform, and no single team's testing can represent every platform.

These rapid changes make collaboration across platforms and organizations increasingly valuable. The best insights come from practitioners running models and workflows on real content and comparing notes. When one team identifies and publishes a new pattern, others can catch it sooner. We encourage practitioners to join us and share what they are finding.

To test with us, bring your policy to our [workshop on October 27](#) and run it across RMC models, including PolicyLM.

Appendix 1 Examples of routing architectures

Live chat and real time text

For a single-model live workflow, prioritize latency, then balanced accuracy and per-language calibration. A stack might include:

- A deterministic keyword or hash precheck for known strings.
- One fast, policy-aware classifier that makes the decision using per-category thresholds, such as PolicyLM-1.7B.
- A larger model for decisions requiring a rationale, such as appeals, and as an LLM-as-a-judge on sampled production data to assess drift.

Pair this with [Roblox's PII classifier](#) for personal information and off-platform steering, and with [Sentinel](#) across users' recent messages to identify rare, severe patterns. Route those patterns to investigators rather than taking automatic action.

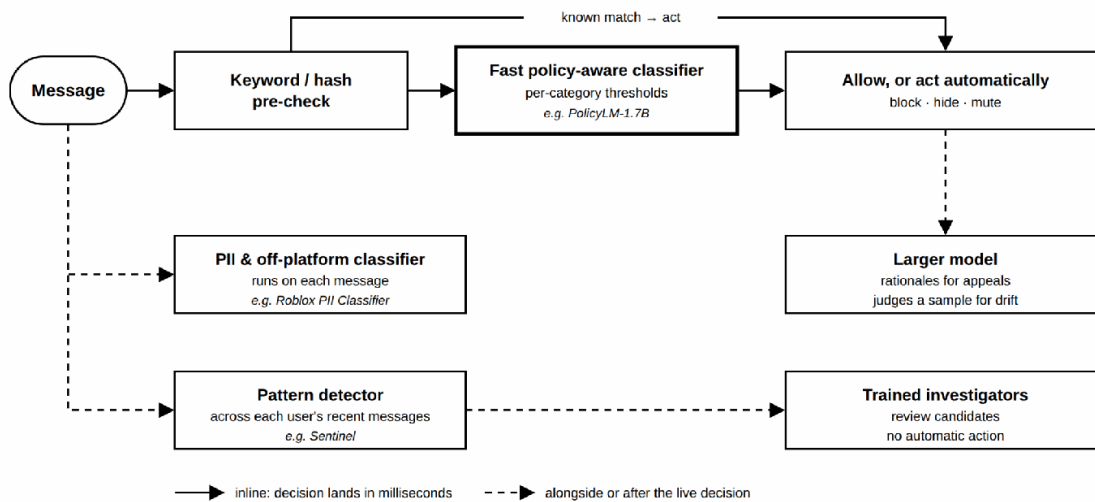


Figure 12. Live chat and real time text

User uploaded images

Prioritize first-stage recall and cost. A stack might include:

- A fast, high-recall image labeler or classifier that runs on every upload.
- Automatic approval when no label triggers.
- A policy-aware LLM that reads a more detailed policy, the labels, and the context when any label triggers, then makes the final decision.

From the RMC, [Shieldstral](#) can serve as the first pass with a recall-tuned threshold. It asks one policy question per call, so keep the first-stage label set small and leave detailed policy review to the second stage. Use [gpt-oss-safeguard](#) as the second stage when you want the reasoning logged.

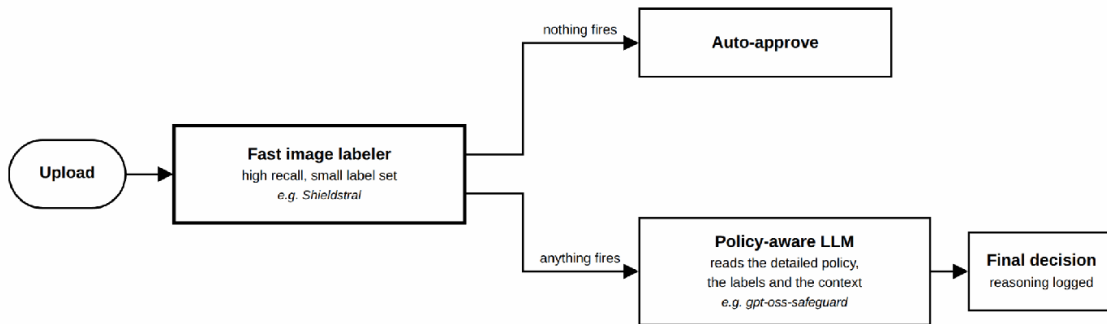


Figure 13. User uploaded images

Asynchronous review of profiles listings and bios

Prioritize accuracy and steerability, because policies change and specialists are tuned separately. A stack might include:

- A fast, high-recall text labeler, such as CoPE, that runs on every item.
- A router policy that sends hard subcategories to specialist policies, potentially with larger models attached.

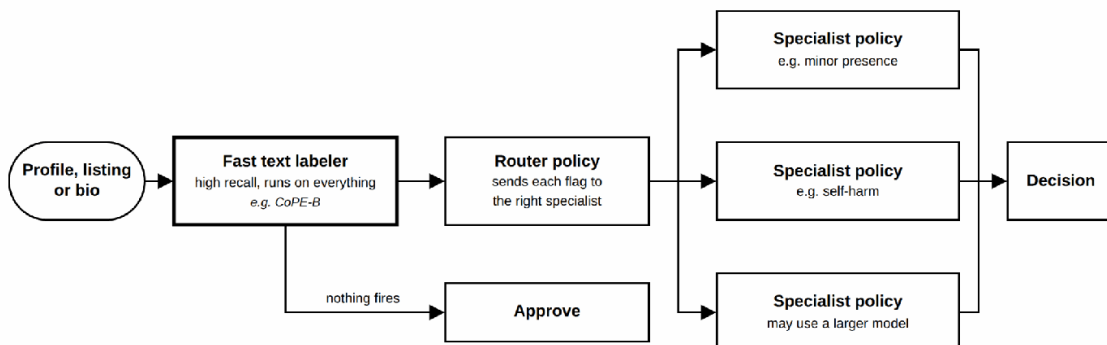


Figure 14. Asynchronous review of profiles listings and bios

Agent to agent safety

Prioritize low cost while nothing is happening and careful review at the moment of consequence. A stack might include:

- A lightweight classifier on every message in the agent transcript.
- A slower reasoning model that runs only when the classifier triggers or the receiving agent is about to take a real-world action, such as spending money, sending content externally, or calling a tool.

From the RMC, an output guardrail such as Mila's could run on every assistant response, with [gpt-oss-safeguard](#) providing the careful check at the action gate. What steerability means when the "policy" includes procedural rules remains an open question.

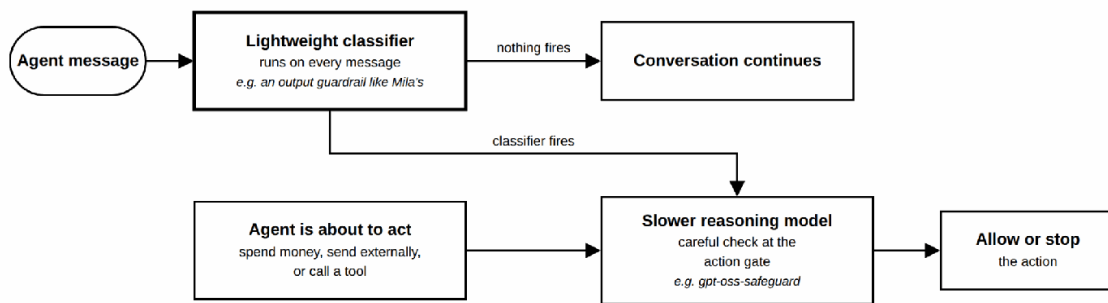


Figure 15. Agent to agent safety

Voice chat

Prioritize latency and language coverage. A stack might include:

- An audio classifier on voice segments, such as Roblox's [Voice-Safety-Classifier-v3](#), which supports segments up to 15 seconds and operating points per language.
- Escalation of flagged segments to a transcript plus a text policy model, or to a human, for rules that go beyond the classifier's fixed categories.

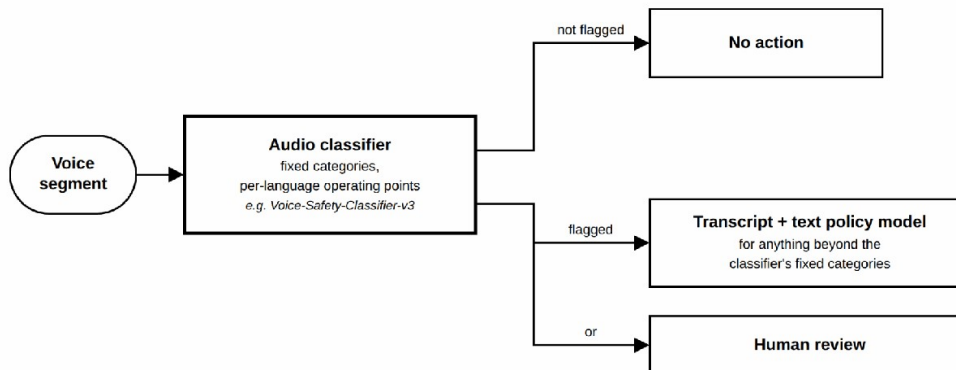


Figure 16. Voice chat

Rare and severe harms

For harms such as grooming patterns, prioritize recall. Examples are scarce, and individual messages often appear harmless. A stack might include:

- A pattern detector such as [Sentinel](#) across each user's recent messages to generate candidates.
- Trained investigators to review those candidates. This tier finds cases; it does not take automatic action.

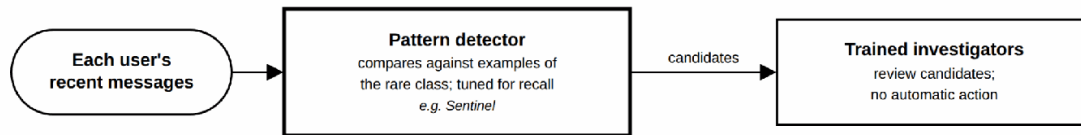


Figure 17. Rare and severe harms

Appendix 2 Model examples from the ROOST Model Community

The descriptions below come from each model's own card. They are not benchmark comparisons.

Model	What it is	Use when	Consider limitations when
PolicyLM-1.7B (Musubi)	1.7B classifier that reads your policy and returns a 0–1 score for multiple categories in one pass. Text only.	You need a low-latency, inexpensive model that gives a balanced decision against your own policy across all policy categories at once.	You need a written rationale, images or audio, very long policies, or policies you invert at inference time.
Shieldstral -1.0-3B (Mistral)	3B policy-adaptive classifier for text, images, or both. One yes/no question per call, with a score you can threshold.	You need an inexpensive, fast first pass on text, images, or mixed media.	You need many categories scored per item, because each policy requires a separate call.
CoPE-B -A4B (Zentropi)	Policy-adaptive classifier with 25.2B total and 3.8B active parameters (mixture of experts), and a single-token 0/1 output.	Your policies are custom, permissive, or unusual, and you want the model to follow them closely.	You need per-category scores for routing, or a hard floor if the policy text is wrong or compromised.
gpt-oss-safeguard (OpenAI, 20B and 120B)	Reasoning model that takes your policy and returns a decision with its reasoning. Adjustable reasoning effort.	Decisions need a written reason, such as appeals, bans, or takedowns; you are labeling offline or building golden sets; or you need a second-stage judge.	The decision must arrive inline in live chat.
Sentinel (Roblox)	Library that uses embeddings to compare messages against examples of a rare class and looks for patterns across a user's recent messages.	You are investigating extremely rare, severe patterns (Roblox built it for child safety), where recall matters most and results go to investigators.	You need a per-message decision or an automatic action.
Voice-Safety-Classifier-v3 (Roblox)	0.3B audio classifier with eight fixed categories, 30 languages, and operating points set per language.	You moderate voice chat and want to classify audio directly.	Your policy does not match its categories.
Roblox-PII-Classifier-v2 (Roblox)	0.6B classifier that reads the chat history around a message: PII solicitation, PII disclosure, and steering users off-platform. Covers 189 languages.	You need to catch personal information and off-platform steering in chat, especially where minors are present.	You need general harm categories.
Mila Suicide-Prevention Output Guardrail (Mila)	0.1B English-only classifier for a chatbot's responses, not user input. Favors recall.	You want an inexpensive check on every chatbot response as one layer of a stacked defense.	You are screening user input, need other languages, or want it as the only decision-maker.

Combining models in practice

These models are often combined:

- A game with voice and text might run the voice classifier on audio, PolicyLM on text chat, the PII classifier on conversations, and [Sentinel](#) across each user's history for rare, severe patterns.
- A marketplace or dating app might use [Shieldstral](#) as a first pass on images, then [gpt-oss-safeguard](#) on flagged items with the full policy.
- An AI chatbot or agent might run an output guardrail such as Mila's on every response, plus a reasoning model before any consequential action.